CS 392 Final Cheat Sheet

Travis Falk • C++ competitive programming reference

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pairll = pair<ll,ll>;

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  // ... solve ...
  return 0;
}
```

Problem Index — alphabetical

10 Kinds of People (Kattis) 7
All Pairs Shortest Path (Kattis) 10
Almost Union-Find (Kattis) 14
Array Description (CSES) 16
Baby Bites (Kattis) 20
Book Shop (CSES) 14
Building Roads (CSES) 5
Building Teams (CSES) 5
Cent Savings (Kattis) 17
Coin Combinations I (CSES) 14
Coin Combinations II (CSES) 14
Conservation (Kattis) 11
Counting Divisors (CSES) 19
Counting Rooms (CSES) 4
Counting Towers (CSES) 16
Course Schedule (CSES) 11
Cycle Finding (CSES) 10
Dice Combinations (CSES) 15
Dynamic Minimum Range Queries (CSES) 18
Dynamic Sum Range Queries (CSES) 18
Edit Distance (CSES) 16
Exact Change (Kattis) 17
Flight Discounts (CSES) 8
Flight Routes Check (CSES) 12
Flowery Trails (Kattis) 8
Game Routes (CSES) 11
GCDs (Kattis) 20
Grandpa Bernie (Kattis) 20
Grid (Kattis) 7
Grid Paths I (CSES) 15
Investigation (CSES) 8
Labyrinth (CSES) 5
Ladice (Kattis) 13
Longest Flight Route (CSES) 11
Lost Map (Kattis) 13
Message Route (CSES) 5
Minimizing Coins (CSES) 15
Modular Arithmetic (Kattis) 20
Money Sums (CSES) 15
Monsters (CSES) 6
Nikola (Kattis) 9
Nine Packs (Kattis) 16
Planets and Kingdoms (CSES) 12
Prime Sieve (Kattis) 19
Range Update Queries (CSES) 19
Rectangle Cutting (CSES) 15
Removing Digits (CSES) 15
Restaurant Orders (Kattis) 17
Road Construction (CSES) 13
Road Reparation (CSES) 13
Round Trip (CSES) 6
Shortest Routes I (CSES) 8
Shortest Routes II (CSES) 10
Static Minimum Range Queries (CSES) 18
Static Sum Range Queries (CSES) 17
Texas Summers (Kattis) 9
Tri Tiling (Kattis) 16
Union-Find (Kattis) 13
Weak Vertices (Kattis) 7
Where's My Internet (Kattis) 7

PART I — Primitives

Vectors

```cpp
// Dynamic array. [] and .at() are O(1).
// push/pop_back O(1). insert/erase mid O(N).
vector<ll> v;                   // empty
vector<ll> a(5);                // 5 zeros
vector<ll> b(5, 10);            // {10,10,10,10,10}
vector<ll> c = {5,2,8,1,9};
vector<ll> d(c.begin(), c.begin()+3);    // {5,2,8}
vector<vector<ll>> grid(n, vector<ll>(m, 0));

v.push_back(7);   v.emplace_back(7);
v.pop_back();
v.insert(v.begin()+2, 99);      // O(N)
v.erase(v.begin()+2);           // remove idx 2
v.erase(v.begin(), v.begin()+2);
v.clear(); v.resize(10); v.resize(10, -1);
fill(v.begin(), v.end(), 0);

ll x = v[0];    ll y = v.at(0);   // .at() throws
ll n = v.size();  bool e = v.empty();
```

Sorting & <algorithm>

```cpp
sort(v.begin(), v.end());                // asc
sort(v.begin(), v.end(), greater<ll>()); // desc
sort(v.begin(), v.end(),
     [](ll a, ll b){ return a > b; });   // custom
reverse(v.begin(), v.end());

auto it  = find(v.begin(), v.end(), 5);
ll   cnt = count(v.begin(), v.end(), 5);
ll   mn  = *min_element(v.begin(), v.end());
ll   mx  = *max_element(v.begin(), v.end());
ll   sum = accumulate(v.begin(), v.end(), 0LL);
iota(v.begin(), v.end(), 0);    // 0,1,2,3,...

// Dedupe: sort + unique + erase
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());

// All permutations (must start sorted)
sort(v.begin(), v.end());
do { /* process v */ }
while (next_permutation(v.begin(), v.end()));
```

Binary Search & Bounds (sorted vector)

```cpp
// 4 shapes -- all O(log N):
//   first >= x :  lower_bound(x)
//   first >  x :  upper_bound(x)
//   last  <  x :  lower_bound(x), --it (skip if begin)
//   last  <= x :  upper_bound(x), --it (skip if begin)

bool found = binary_search(v.begin(), v.end(), 5);
auto lb = lower_bound(v.begin(), v.end(), 4);
auto ub = upper_bound(v.begin(), v.end(), 4);

// Once you have a bound iterator:
if (lb == v.end()) { /* no match */ }
ll val = *lb;
ll idx = lb - v.begin();   // O(1) for vector
*lb = 99;                  // mutate (may break sort!)
```

```
v.erase(lb);                // O(N), invalidates lb
```

Queue, Stack, Priority Queue

```
queue<ll> q;  q.push(1); q.push(2);
ll f = q.front();  ll b = q.back();
q.pop();  q.size();  q.empty();

stack<ll> s;  s.push(1); s.push(2);
if (!s.empty()) { ll t = s.top(); s.pop(); }

// Max-heap (default)
priority_queue<ll> pq;
pq.push(10); pq.push(20);
ll top = pq.top(); pq.pop();          // 20

// Min-heap (3 template args required)
priority_queue<ll, vector<ll>, greater<ll>> minpq;

// PQ of pairs -- lex compare by first, then second.
priority_queue<pairll> pqp;          // max by first
priority_queue<pairll, vector<pairll>,
               greater<pairll>> pqmin;  // Dijkstra shape

// PQ with custom comparator (struct, not lambda)
struct Cmp {
  bool operator()(const pairll& a, const pairll& b)
    const { return a.second > b.second; } // min by 2nd
};
priority_queue<pairll, vector<pairll>, Cmp> pqc;
```

Maps (ordered & unordered)

```
// map: red-black tree, O(log N), needs <.
// unordered_map: hash, O(1) avg / O(N) worst.
map<string,ll> m;
unordered_map<string,ll> um;

m["apple"] = 5;              // creates if absent
ll v = m["apple"];           // WARNING: creates!
m.insert({"banana", 3});     // no-op if present
m.emplace("cherry", 7);
m.erase("apple");

// Safe existence (does NOT insert)
if (m.count("apple")) { }
if (m.find("apple") != m.end()) { }

for (auto& [k,v] : m) { }   // sorted for map

// Ordered-map binary search (NOT on unordered):
auto it = m.lower_bound(key);  // first >= key
auto jt = m.upper_bound(key);  // first >  key
// Once you have it:  it->first (key), it->second (val)
// it->second = x; is OK; key is const.
// next(it), prev(it) for neighbors.
// Min key: m.begin()->first; max: m.rbegin()->first.
```

Sets (set, multiset, unordered_set)

```
set<ll> s = {3,1,4,1,5};     // {1,3,4,5}
multiset<ll> ms = {1,1,2};  // {1,1,2} dupes OK
unordered_set<ll> us;

s.insert(2); s.erase(3);     // O(log N)
ms.erase(1);                 // removes ALL 1s
ms.erase(ms.find(1));        // removes ONE 1

bool has = s.count(2);       // 0 or 1 for set
ll freq = ms.count(1);       // real count

// Ordered binary search (NOT on unordered):
auto it = s.lower_bound(4);  // first >= 4
auto jt = s.upper_bound(4);  // first >  4
// Predecessor (last < x):
//   auto p = s.lower_bound(x);
//   if (p != s.begin()) --p;

*s.begin();    // min
*s.rbegin();   // max
// Values read-only thru iterator (would break order).
```

Strings

```
string s = "hello";
string t(5, 'a');              // "aaaaa"
// getline(cin, s);            // whole line
s += " world";  s.push_back('!');  s.pop_back();
s.insert(5, "XYZ");
s.erase(0, 3);                 // erase 3 chars @ idx 0
s.replace(0, 5, "HELLO");
reverse(s.begin(), s.end());

string sub = s.substr(0, 4);   // (start, length)
string suf = s.substr(4);      // to end

size_t p = s.find("lo");       // or npos
size_t r = s.rfind("lo");      // last occurrence
if (p != string::npos) { }

ll   i = stoll("12345678901");
double d = stod("3.14");
string sn = to_string(42);
// <cctype>: isdigit isalpha isalnum isspace
//           islower isupper tolower toupper
```

Pairs & Tuples

```
pair<ll,string> p = {1, "hi"};
auto [a,b] = p;                // structured binding

tuple<ll,double,string> t = {1, 3.14, "Pi"};
ll i = get<0>(t);
auto [A,B,C] = t;              // preferred
tie(i, /*...*/) = t;           // unpack into vars

// Both compare LEXICOGRAPHICALLY.
vector<pairll> vp = {{3,1},{1,5},{2,2}};
sort(vp.begin(), vp.end());    // {{1,5},{2,2},{3,1}}
```

Disjoint Set Union (Union-Find)

```
// find + unite: O(alpha(n)) ~ O(1) amortized.
struct DSU {
  vector<ll> parent, sz;
  DSU(ll n) : parent(n), sz(n, 1) {
    iota(parent.begin(), parent.end(), 0);
  }
  ll find(ll i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]);   // path compr.
  }
  bool unite(ll i, ll j) {
    ll ri = find(i), rj = find(j);
    if (ri == rj) return false;
    if (sz[ri] < sz[rj]) swap(ri, rj);    // union by sz
    parent[rj] = ri;
    sz[ri] += sz[rj];
```

```
    return true;
  }
};
```

Segment Tree (point update, range sum)

```
// build O(N), update O(log N), query [l,r) O(log N).
// Iterative; 1-indexed internally; tree size = 2n.
class SegTree {
public:
  vector<ll> tree;
  ll n, sz;
  SegTree(vector<ll>& ar) {
    sz = ar.size();
    n = 1;
    while (n < sz) n <<= 1;
    tree.assign(2*n, 0);
    for (ll i = 0; i < sz; i++) tree[i+n] = ar[i];
    for (ll i = n-1; i > 0; i--)
      tree[i] = tree[2*i] + tree[2*i+1];
  }
  void update(ll i, ll val) {
    i += n;  tree[i] = val;
    while (i > 1) { i /= 2; tree[i] = tree[2*i] + tree[2*i
    +1]; }
  }
  ll query(ll l, ll r) {   // sum over [l, r)
    ll ans = 0;
    l += n; r += n;
    while (l < r) {
      if (l & 1) ans += tree[l++];
      if (r & 1) ans += tree[--r];
      l >>= 1;  r >>= 1;
    }
    return ans;
  }
};
```

Sieve of Eratosthenes (smallest prime factor)

```
// sieve[x] = 0 if x prime, else smallest prime factor.
// O(n log log n).
vector<ll> sieve(n+1, 0);
for (ll x = 2; x <= n; x++) {
  if (sieve[x]) continue;              // x is prime
  for (ll u = 2*x; u <= n; u += x)
    if (!sieve[u]) sieve[u] = x;
}
// To factor k: while (sieve[k]) { p = sieve[k]; k/=p; }
//              then k itself is a prime factor (if >1).
```

BFS (unweighted shortest path)

```
// O(V + E). dist[i] = hops from src, -1 if unreachable.
vector<ll> bfs(ll src, ll n, vector<vector<ll>>& adj) {
  vector<ll> dist(n+1, -1);
  queue<ll> q;
  dist[src] = 0;  q.push(src);
  while (!q.empty()) {
    ll u = q.front(); q.pop();
    for (ll v : adj[u]) if (dist[v] == -1) {
      dist[v] = dist[u] + 1;
      q.push(v);
    }
  }
  return dist;
}
```

DFS (iterative & recursive)

```
// Iterative: component detection, reachability.
vector<bool> dfs_it(ll src, ll n, vector<vector<ll>>& adj)
     {
  vector<bool> vis(n+1, false);
  stack<ll> st;  st.push(src);
  while (!st.empty()) {
    ll u = st.top(); st.pop();
    if (vis[u]) continue;
    vis[u] = true;
    for (ll v : adj[u]) if (!vis[v]) st.push(v);
  }
  return vis;
}

// Recursive: trees, backtracking, coloring.
vector<bool> vis;   // global to avoid passing
void dfs(ll u, vector<vector<ll>>& adj) {
  vis[u] = true;
  for (ll v : adj[u]) if (!vis[v]) dfs(v, adj);
}
// Usage:  vis.assign(n+1, false);  dfs(src, adj);
```

Dijkstra (non-negative weights)

```
// O((V+E) log V). adj[u] = {(v, w), ...}.
vector<ll> dijkstra(ll src, ll n,
                    vector<vector<pairll>>& adj) {
  const ll INF = 1e18;
  vector<ll> dist(n+1, INF);
  priority_queue<pairll, vector<pairll>,
                 greater<pairll>> pq;     // min-heap
  dist[src] = 0;
  pq.push({0, src});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;            // stale
    for (auto [v, w] : adj[u]) {
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        pq.push({dist[v], v});
      }
    }
  }
  return dist;
}
```

DP: 0/1 Knapsack (1D)

```
// dp[j] = max value at capacity j.
// Items OUTER, capacity BACKWARD (prevents re-use).
vector<ll> dp(W+1, 0);
for (ll i = 0; i < n; i++) {
  for (ll j = W; j >= weight[i]; j--) {
    dp[j] = max(dp[j], dp[j-weight[i]] + value[i]);
  }
}
cout << dp[W];
```

DP: Unbounded Knapsack (1D)

```
// Each item takeable unlimited times.
// Items OUTER, capacity FORWARD (allows re-use).
vector<ll> dp(W+1, 0);
for (ll i = 0; i < n; i++) {
  for (ll j = weight[i]; j <= W; j++) {
    dp[j] = max(dp[j], dp[j-weight[i]] + value[i]);
  }
}
```

DP: Bounded Knapsack (binary decomp → 0/1)

```
// copies[i] copies of item i. Decompose into bundles
// of sizes 1,2,4,...,rem which sum to copies[i].
// Any count 0..copies[i] = subset sum of bundles.
vector<pairll> bundles;  // (weight, value)
for (ll i = 0; i < n; i++) {
  ll k = copies[i];
  for (ll t = 1; t <= k; t *= 2) {
    bundles.push_back({t*weight[i], t*value[i]});
    k -= t;
  }
  if (k > 0)
    bundles.push_back({k*weight[i], k*value[i]});
}
// Then standard 0/1 knapsack over bundles.
vector<ll> dp(W+1, 0);
for (auto [w, v] : bundles) {
  for (ll j = W; j >= w; j--) {
    dp[j] = max(dp[j], dp[j-w] + v);
  }
}
```

DP: Counting (ordered composition)

```
// # ways to make sum n using parts from S, order matters.
// e.g. dice: S = {1,2,3,4,5,6}.  1+2 != 2+1.
// Target OUTER, parts INNER.
const ll MOD = 1000000007;
vector<ll> dp(n+1, 0);
dp[0] = 1;
for (ll s = 1; s <= n; s++) {
  for (ll k : {1,2,3,4,5,6}) {
    if (s - k >= 0)
      dp[s] = (dp[s] + dp[s-k]) % MOD;
  }
}
cout << dp[n];
```

DP: Counting (unordered – coin change)

```
// # ways to make sum n using coins in C, order irrelevant
    .
// 1+2 and 2+1 counted ONCE.
// Coins OUTER, target INNER.  (swap vs ordered version)
vector<ll> dp(n+1, 0);
dp[0] = 1;
for (ll c : coins) {
  for (ll s = c; s <= n; s++) {
    dp[s] += dp[s - c];
  }
}
```

DP: Longest Common Subsequence

```
// dp[i][j] = LCS length of a[0..i) and b[0..j).
ll n = a.size(), m = b.size();
vector<vector<ll>> dp(n+1, vector<ll>(m+1, 0));
for (ll i = 1; i <= n; i++) {
  for (ll j = 1; j <= m; j++) {
    if (a[i-1] == b[j-1])
      dp[i][j] = dp[i-1][j-1] + 1;
    else
      dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
  }
}
cout << dp[n][m];
```

DP: Edit Distance (Levenshtein)

```
// dp[i][j] = min edits to make a[0..i) == b[0..j).
vector<vector<ll>> dp(n+1, vector<ll>(m+1));
for (ll i = 0; i <= n; i++) dp[i][0] = i;  // deletions
for (ll j = 0; j <= m; j++) dp[0][j] = j;  // insertions
for (ll i = 1; i <= n; i++) {
  for (ll j = 1; j <= m; j++) {
    if (a[i-1] == b[j-1])
      dp[i][j] = dp[i-1][j-1];
    else
      dp[i][j] = 1 + min({dp[i-1][j],       // delete
                          dp[i][j-1],       // insert
                          dp[i-1][j-1]});   // replace
  }
}
```

DP Knapsack-Family Direction Table

```
// Problem          | Outer  | Inner     | Note
// --------------- | ------ | --------- |
    -----------------
// 0/1 knapsack     | items  | cap BACK  | each used <= 1x
// Unbounded knap   | items  | cap FWD   | unlimited uses
// Bounded knap     | bundles| cap BACK  | decompose first
// Count ordered    | target | parts     | 1+2 != 2+1
// Count unordered  | coins  | target    | 1+2 == 2+1
//
// Mental model when collapsing 2D -> 1D:
//   "previous row" lives in not-yet-overwritten cells.
//   BACKWARD iter: reads old row's smaller index.
//   FORWARD iter: reads THIS row's smaller index.
```

PART II — Graphs: BFS / DFS / Grids

Counting Rooms (CSES). Grid of '.' floor / '#' wall; count connected floor regions via flood-fill BFS.

```
int n, m;
cin >> n >> m;

vector<vector<char>> grid(n, vector<char>(m));
for (int i = 0; i < n; i++)
  for (int j = 0; j < m; j++)
    cin >> grid[i][j];

int dr[] = {1, -1, 0, 0};
int dc[] = {0, 0, 1, -1};

auto bfs = [&](int si, int sj) {
  queue<pair<int,int>> q;
  grid[si][sj] = '#';
  q.push({si, sj});
  while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
      int nr = r + dr[d], nc = c + dc[d];
      if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
      if (grid[nr][nc] != '.') continue;
      grid[nr][nc] = '#';
      q.push({nr, nc});
    }
  }
};

int count = 0;
for (int i = 0; i < n; i++)
  for (int j = 0; j < m; j++)
    if (grid[i][j] == '.') { bfs(i, j); count++; }
```

```
cout << count;
```

Labyrinth (CSES). BFS from A to B in a grid; reconstruct the move-string (UDLR) via parent pointers.

```
int n, m;
cin >> n >> m;

vector<string> grid(n);
pair<int,int> A, B;
for (int i = 0; i < n; i++) {
  cin >> grid[i];
  for (int j = 0; j < m; j++) {
    if (grid[i][j] == 'A') A = {i, j};
    if (grid[i][j] == 'B') B = {i, j};
  }
}

int dr[] = {1, -1, 0, 0};
int dc[] = {0, 0, -1, 1};
char dn[] = {'D', 'U', 'L', 'R'};

vector<vector<pair<int,int>>> par(n, vector<pair<int,int>>(m,
     {-1, -1}));
queue<pair<int,int>> q;
q.push(A);
grid[A.first][A.second] = '#';

bool found = (A == B);
while (!q.empty() && !found) {
  auto [r, c] = q.front(); q.pop();
  for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
    if (grid[nr][nc] == '#') continue;
    par[nr][nc] = {r, c};
    if (make_pair(nr, nc) == B) { found = true; break; }
    grid[nr][nc] = '#';
    q.push({nr, nc});
  }
}

if (!found) { cout << "NO"; return 0; }

string path;
pair<int,int> cur = B;
while (cur != A) {
  auto p = par[cur.first][cur.second];
  for (int d = 0; d < 4; d++)
    if (p.first + dr[d] == cur.first && p.second + dc[d] == cur.
     second) {
      path += dn[d]; break;
    }
  cur = p;
}
reverse(path.begin(), path.end());

cout << "YES\n" << path.size() << "\n" << path;
```

Message Route (CSES). Undirected unweighted BFS 1→n; print path length and node sequence.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  adj[b].push_back(a);
}

vector<int> par(n + 1, -1), dist(n + 1, -1);
queue<int> q;
q.push(1);
dist[1] = 0;

while (!q.empty()) {
  int u = q.front(); q.pop();
  for (int v : adj[u]) {
    if (dist[v] == -1) {
      dist[v] = dist[u] + 1;
      par[v] = u;
      q.push(v);
    }
  }
}

if (dist[n] == -1) { cout << "IMPOSSIBLE"; return 0; }

vector<int> path;
for (int c = n; c != -1; c = par[c]) path.push_back(c);
reverse(path.begin(), path.end());

cout << path.size() << "\n";
for (int x : path) cout << x << " ";
```

Building Roads (CSES). Count connected components with DFS; output the edges needed to link one representative from each component.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  adj[b].push_back(a);
}

vector<bool> vis(n + 1, false);
auto dfs = [&](int s) {
  stack<int> st;
  st.push(s);
  while (!st.empty()) {
    int u = st.top(); st.pop();
    if (vis[u]) continue;
    vis[u] = true;
    for (int v : adj[u])
      if (!vis[v]) st.push(v);
  }
};

vector<int> reps;
for (int i = 1; i <= n; i++)
  if (!vis[i]) { reps.push_back(i); dfs(i); }

cout << reps.size() - 1 << "\n";
for (int i = 0; i + 1 < (int)reps.size(); i++)
  cout << reps[i] << " " << reps[i + 1] << "\n";
```

Building Teams (CSES). Check bipartite: BFS 2-color each component; report IMPOSSIBLE if two same-color neighbors.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  adj[b].push_back(a);
}

vector<int> team(n + 1, 0);
for (int s = 1; s <= n; s++) {
  if (team[s]) continue;
  queue<int> q;
  q.push(s);
  team[s] = 1;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) {
      if (!team[v]) {
        team[v] = 3 - team[u];
        q.push(v);
      } else if (team[v] == team[u]) {
        cout << "IMPOSSIBLE"; return 0;
      }
    }
  }
}

for (int i = 1; i <= n; i++) cout << team[i] << " ";
```

Round Trip (CSES). Find any cycle in an undirected graph with DFS; a back-edge to a visited non-parent closes the cycle.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  adj[b].push_back(a);
}

vector<int> par(n + 1, 0);
vector<bool> vis(n + 1, false);
int cycle_start = -1, cycle_end = -1;

function<bool(int,int)> dfs = [&](int u, int p) -> bool {
  vis[u] = true;
  for (int v : adj[u]) {
    if (v == p) continue;
    if (vis[v]) {
      cycle_start = v; cycle_end = u;
      return true;
    }
    par[v] = u;
    if (dfs(v, u)) return true;
  }
  return false;
};

for (int i = 1; i <= n; i++) {
  if (!vis[i] && dfs(i, -1)) {
    vector<int> cycle = {cycle_start};
    for (int v = cycle_end; v != cycle_start; v = par[v])
      cycle.push_back(v);
    cycle.push_back(cycle_start);
    cout << cycle.size() << "\n";
    for (int v : cycle) cout << v << " ";
    return 0;
  }
}
cout << "IMPOSSIBLE";
```

Monsters (CSES). Multi-source BFS from every M to get `md`; then BFS from A only entering cells where `pd<md`; exit on first boundary cell.

```
int n, m;
cin >> n >> m;
vector<string> grid(n);
for (auto& r : grid) cin >> r;

int dr[] = {1, -1, 0, 0};
int dc[] = {0, 0, -1, 1};
char dn[] = {'D', 'U', 'L', 'R'};

// Phase 1: multi-source BFS from all monsters
vector<vector<int>> md(n, vector<int>(m, INT_MAX));
queue<pair<int,int>> q;
int sr = -1, sc = -1;
for (int i = 0; i < n; i++)
  for (int j = 0; j < m; j++) {
    if (grid[i][j] == 'M') { md[i][j] = 0; q.push({i, j}); }
    else if (grid[i][j] == 'A') { sr = i; sc = j; }
  }
while (!q.empty()) {
  auto [r, c] = q.front(); q.pop();
  for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
    if (grid[nr][nc] == '#' || md[nr][nc] != INT_MAX) continue;
    md[nr][nc] = md[r][c] + 1;
    q.push({nr, nc});
  }
}

// Phase 2: BFS from player, only through cells reached strictly
      first
vector<vector<int>> pd(n, vector<int>(m, INT_MAX));
vector<vector<pair<int,int>>> par(n, vector<pair<int,int>>(m,
     {-1, -1}));
pd[sr][sc] = 0;
q.push({sr, sc});
int er = -1, ec = -1;
auto edge = [&](int r, int c) {
  return r == 0 || r == n - 1 || c == 0 || c == m - 1;
};
if (edge(sr, sc)) { er = sr; ec = sc; }

while (!q.empty() && er == -1) {
  auto [r, c] = q.front(); q.pop();
  for (int d = 0; d < 4; d++) {
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
    if (grid[nr][nc] == '#' || pd[nr][nc] != INT_MAX) continue;
    int np = pd[r][c] + 1;
    if (np >= md[nr][nc]) continue;
    pd[nr][nc] = np;
    par[nr][nc] = {r, c};
    q.push({nr, nc});
    if (edge(nr, nc)) { er = nr; ec = nc; break; }
  }
}
```

```
if (er == -1) { cout << "NO"; return 0; }

string path;
int r = er, c = ec;
while (r != sr || c != sc) {
  auto [pr, pc] = par[r][c];
  for (int d = 0; d < 4; d++)
    if (pr + dr[d] == r && pc + dc[d] == c) { path += dn[d];
     break; }
  r = pr; c = pc;
}
reverse(path.begin(), path.end());

cout << "YES\n" << path.size() << "\n" << path;
```

Where's My Internet (Kattis). BFS reachability from house 1; list houses that are *not* reached.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  adj[b].push_back(a);
}

vector<bool> vis(n + 1, false);
queue<int> q;
q.push(1);
vis[1] = true;

while (!q.empty()) {
  int u = q.front(); q.pop();
  for (int v : adj[u])
    if (!vis[v]) { vis[v] = true; q.push(v); }
}

vector<int> unconnected;
for (int i = 2; i <= n; i++)
  if (!vis[i]) unconnected.push_back(i);

if (unconnected.empty()) cout << "Connected";
else for (int x : unconnected) cout << x << "\n";
```

10 Kinds of People (Kattis). Label each grid cell with its connected-component id (same digit, 4-dir); queries compare ids.

```
int n, m;
cin >> n >> m;
vector<string> grid(n);
for (auto& r : grid) cin >> r;

vector<vector<int>> comp(n, vector<int>(m, -1));
int dr[] = {0, 0, 1, -1};
int dc[] = {1, -1, 0, 0};
int id = 0;

auto bfs = [&](int sr, int sc) {
  queue<pair<int,int>> q;
  q.push({sr, sc});
  comp[sr][sc] = id;
  char ch = grid[sr][sc];
  while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
      int nr = r + dr[d], nc = c + dc[d];
      if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
      if (comp[nr][nc] != -1 || grid[nr][nc] != ch) continue;
      comp[nr][nc] = id;
      q.push({nr, nc});
    }
  }
};

for (int i = 0; i < n; i++)
  for (int j = 0; j < m; j++)
    if (comp[i][j] == -1) { bfs(i, j); id++; }

int q;
cin >> q;
while (q--) {
  int r1, c1, r2, c2;
  cin >> r1 >> c1 >> r2 >> c2;
  r1--; c1--; r2--; c2--;
  if (comp[r1][c1] == comp[r2][c2])
    cout << (grid[r1][c1] == '0' ? "binary" : "decimal") << "\n"
     ;
  else
    cout << "neither\n";
}
```

Grid (Kattis). BFS on grid where each cell's digit d is the step size in the 4 cardinal directions.

```
int n, m;
cin >> n >> m;
vector<string> grid(n);
for (auto& r : grid) cin >> r;

vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int,int>> q;
q.push({0, 0});
dist[0][0] = 0;

int dr[] = {0, 0, 1, -1};
int dc[] = {1, -1, 0, 0};

while (!q.empty()) {
  auto [r, c] = q.front(); q.pop();
  int d = grid[r][c] - '0';
  if (d == 0) continue;
  for (int k = 0; k < 4; k++) {
    int nr = r + dr[k] * d;
    int nc = c + dc[k] * d;
    if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
    if (dist[nr][nc] != -1) continue;
    dist[nr][nc] = dist[r][c] + 1;
    q.push({nr, nc});
  }
}

cout << dist[n - 1][m - 1];
```

Weak Vertices (Kattis). Adjacency matrix; a vertex is weak if it is *not* part of any triangle. Brute triple loop.

```
int n;
cin >> n;
while (n != -1) {
  vector<vector<int>> adj(n, vector<int>(n));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      cin >> adj[i][j];

  for (int i = 0; i < n; i++) {
```

```
    bool inTriangle = false;
    for (int j = 0; j < n && !inTriangle; j++)
      for (int k = j + 1; k < n && !inTriangle; k++)
        if (adj[i][j] && adj[i][k] && adj[j][k])
          inTriangle = true;
    if (!inTriangle) cout << i << " ";
  }
  cout << "\n";
  cin >> n;
}
```

Graphs: Dijkstra & SSSP Variants

Shortest Routes I (CSES). Single-source Dijkstra on weighted directed graph; print $d(1, i)$ for all i.

```
int n, m;
cin >> n >> m;

vector<vector<pair<int,ll>>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  ll c;
  cin >> a >> b >> c;
  adj[a].push_back({b, c});
}

const ll INF = 1e18;
vector<ll> dist(n + 1, INF);
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq
    ;
dist[1] = 0;
pq.push({0, 1});

while (!pq.empty()) {
  auto [d, u] = pq.top(); pq.pop();
  if (d > dist[u]) continue;          // stale
  for (auto [v, w] : adj[u]) {
    if (dist[u] + w < dist[v]) {
      dist[v] = dist[u] + w;
      pq.push({dist[v], v});
    }
  }
}

for (int i = 1; i <= n; i++) cout << dist[i] << " ";
```

Flight Discounts (CSES). Dijkstra over state (node, coupon_used$\in \{0, 1\}$); the coupon halves exactly one edge.

```
int n, m;
cin >> n >> m;

vector<vector<pair<int,ll>>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  ll c;
  cin >> a >> b >> c;
  adj[a].push_back({b, c});
}

const ll INF = 1e18;
// dist[u][0] = shortest to u with coupon still available
// dist[u][1] = shortest to u with coupon already spent
vector<vector<ll>> dist(n + 1, vector<ll>(2, INF));
priority_queue<tuple<ll,int,int>,
               vector<tuple<ll,int,int>>, greater<>> pq;
dist[1][0] = 0;
pq.push({0, 1, 0});

while (!pq.empty()) {
  auto [d, u, used] = pq.top(); pq.pop();
  if (d > dist[u][used]) continue;
  for (auto [v, w] : adj[u]) {
    // Option 1: don't use coupon on this edge
    if (dist[u][used] + w < dist[v][used]) {
      dist[v][used] = dist[u][used] + w;
      pq.push({dist[v][used], v, used});
    }
    // Option 2: use coupon on this edge (only if still
     available)
    if (used == 0 && dist[u][0] + w / 2 < dist[v][1]) {
      dist[v][1] = dist[u][0] + w / 2;
      pq.push({dist[v][1], v, 1});
    }
  }
}

cout << dist[n][1];
```

Investigation (CSES). Dijkstra while tracking #shortest paths (mod), min-edges, max-edges along any shortest path.

```
int n, m;
cin >> n >> m;

vector<vector<pair<int,ll>>> adj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  ll c;
  cin >> a >> b >> c;
  adj[a].push_back({b, c});
}

const ll INF = 1e18, MOD = 1e9 + 7;
vector<ll> dist(n + 1, INF), ways(n + 1, 0);
vector<int> minE(n + 1, 0), maxE(n + 1, 0);

priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq
    ;
dist[1] = 0;
ways[1] = 1;
pq.push({0, 1});

while (!pq.empty()) {
  auto [d, u] = pq.top(); pq.pop();
  if (d > dist[u]) continue;
  for (auto [v, w] : adj[u]) {
    ll nd = dist[u] + w;
    if (nd < dist[v]) {
      dist[v] = nd;
      ways[v] = ways[u];
      minE[v] = minE[u] + 1;
      maxE[v] = maxE[u] + 1;
      pq.push({nd, v});
    } else if (nd == dist[v]) {
      ways[v] = (ways[v] + ways[u]) % MOD;
      minE[v] = min(minE[v], minE[u] + 1);
      maxE[v] = max(maxE[v], maxE[u] + 1);
    }
  }
}

cout << dist[n] << " " << ways[n] << " "
     << minE[n] << " " << maxE[n];
```

Flowery Trails (Kattis). Run Dijkstra from 0 and from $n - 1$;

an edge (u, v, w) lies on a shortest path iff $d_s[u] + w + d_t[v]$ equals the total. Sum those w and double.

```
int n, m;
cin >> n >> m;

struct Edge { int u, v; ll w; };
vector<Edge> edges(m);
vector<vector<pair<int,ll>>> adj(n);
for (int i = 0; i < m; i++) {
  int u, v;
  ll w;
  cin >> u >> v >> w;
  edges[i] = {u, v, w};
  adj[u].push_back({v, w});
  adj[v].push_back({u, w});
}

auto dijkstra = [&](int src) {
  const ll INF = 1e18;
  vector<ll> d(n, INF);
  priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>>
     pq;
  d[src] = 0;
  pq.push({0, src});
  while (!pq.empty()) {
    auto [dd, u] = pq.top(); pq.pop();
    if (dd > d[u]) continue;
    for (auto [v, w] : adj[u])
      if (d[u] + w < d[v]) {
        d[v] = d[u] + w;
        pq.push({d[v], v});
      }
  }
  return d;
};

vector<ll> ds = dijkstra(0);       // from entrance
vector<ll> dt = dijkstra(n - 1);   // from peak
ll shortest = ds[n - 1];

ll total = 0;
for (auto& e : edges) {
  if (e.u == e.v) continue;        // skip self-loops
  if (ds[e.u] + e.w + dt[e.v] == shortest ||
      ds[e.v] + e.w + dt[e.u] == shortest)
    total += e.w;
}

cout << total * 2;                 // round-trip
```

Texas Summers (Kattis). Dijkstra over complete graph of points; edge cost is squared Euclidean distance; print intermediate shady spots only.

```
int n;
cin >> n;

// Index 0 = dorm, 1..n = shady spots, n+1 = class
vector<pair<ll,ll>> pts(n + 2);
for (int i = 1; i <= n; i++) cin >> pts[i].first >> pts[i].
     second;
cin >> pts[0].first >> pts[0].second;
cin >> pts[n + 1].first >> pts[n + 1].second;
int total = n + 2;

auto d2 = [&](int i, int j) -> ll {
  ll dx = pts[i].first  - pts[j].first;
  ll dy = pts[i].second - pts[j].second;
  return dx * dx + dy * dy;
};

const ll INF = 1e18;
vector<ll> dist(total, INF);
vector<int> prev(total, -1);
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq
     ;
dist[0] = 0;
pq.push({0, 0});

while (!pq.empty()) {
  auto [d, u] = pq.top(); pq.pop();
  if (d > dist[u]) continue;
  for (int v = 0; v < total; v++) {
    if (v == u) continue;
    ll nd = dist[u] + d2(u, v);
    if (nd < dist[v]) {
      dist[v] = nd;
      prev[v] = u;
      pq.push({nd, v});
    }
  }
}

// Reconstruct dorm -> ... -> class; print only shady spot
     indices
vector<int> path;
for (int cur = n + 1; cur != 0; cur = prev[cur]) path.push_back(
     cur);
reverse(path.begin(), path.end());

bool any = false;
for (int x : path)
  if (x >= 1 && x <= n) { cout << (x - 1) << "\n"; any = true; }
if (!any) cout << "-\n";
```

Nikola (Kattis). Dijkstra on state (pos, k); forward jump $k+1$ squares (k becomes $k+1$), backward jump k squares (k unchanged).

```
int n;
cin >> n;
vector<int> cost(n + 1);
for (int i = 1; i <= n; i++) cin >> cost[i];

int maxK = n;
const int INF = INT_MAX / 2;
vector<vector<int>> dist(n + 1, vector<int>(maxK + 1, INF));

priority_queue<tuple<int,int,int>,
               vector<tuple<int,int,int>>, greater<>> pq;
dist[1][0] = 0;
pq.push({0, 1, 0});

while (!pq.empty()) {
  auto [d, pos, k] = pq.top(); pq.pop();
  if (d > dist[pos][k]) continue;

  // Forward: jump size k+1
  int nk = k + 1, fwd = pos + nk;
  if (fwd <= n && nk <= maxK) {
    int nd = d + cost[fwd];
    if (nd < dist[fwd][nk]) {
      dist[fwd][nk] = nd;
      pq.push({nd, fwd, nk});
    }
  }

  // Backward: jump size k (unchanged)
```

```
  if (k > 0) {
    int bwd = pos - k;
    if (bwd >= 1) {
      int nd = d + cost[bwd];
      if (nd < dist[bwd][k]) {
        dist[bwd][k] = nd;
        pq.push({nd, bwd, k});
      }
    }
  }
}

int ans = INF;
for (int k = 1; k <= maxK; k++) ans = min(ans, dist[n][k]);
cout << ans;
```

Graphs: Topological / SCC / All-Pairs / Negative

Shortest Routes II (CSES). All-pairs Floyd-Warshall on undirected graph; answer each query $d(a, b)$ or -1.

```
int n, m, q;
cin >> n >> m >> q;

const ll INF = 1e18;
vector<vector<ll>> dist(n + 1, vector<ll>(n + 1, INF));
for (int i = 1; i <= n; i++) dist[i][i] = 0;

for (int i = 0; i < m; i++) {
  int a, b;
  ll c;
  cin >> a >> b >> c;
  dist[a][b] = min(dist[a][b], c);   // handle parallel edges
  dist[b][a] = min(dist[b][a], c);
}

// Floyd-Warshall: try every node k as an intermediate
for (int k = 1; k <= n; k++)
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
      if (dist[i][k] != INF && dist[k][j] != INF)
        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

while (q--) {
  int a, b;
  cin >> a >> b;
  cout << (dist[a][b] == INF ? -1 : dist[a][b]) << "\n";
}
```

All Pairs Shortest Path (Kattis). Floyd-Warshall that handles negative cycles: after FW, if $d[k][k] < 0$ propagate $-\infty$ to any i, j with a path through k.

```
int n, m, q;
while (cin >> n >> m >> q && (n || m || q)) {
  const ll INF = 1e15;
  vector<vector<ll>> d(n, vector<ll>(n, INF));
  for (int i = 0; i < n; i++) d[i][i] = 0;

  for (int i = 0; i < m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    d[u][v] = min(d[u][v], w);
  }

  // Standard Floyd-Warshall
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (d[i][k] != INF && d[k][j] != INF)
          d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

  // Propagate negative cycles
  for (int k = 0; k < n; k++)
    if (d[k][k] < 0)
      for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
          if (d[i][k] != INF && d[k][j] != INF)
            d[i][j] = -INF;

  while (q--) {
    int u, v;
    cin >> u >> v;
    if      (d[u][v] ==  INF) cout << "Impossible\n";
    else if (d[u][v] == -INF) cout << "-Infinity\n";
    else                      cout << d[u][v] << "\n";
  }
  cout << "\n";
}
```

Cycle Finding (CSES). Bellman-Ford n iterations; if an edge still relaxes on the n-th, a negative cycle exists. Walk parent n steps to land in the cycle, then collect.

```
int n, m;
cin >> n >> m;

struct Edge { int a, b; ll c; };
vector<Edge> edges(m);
for (auto& e : edges) cin >> e.a >> e.b >> e.c;

// Virtual source connected to all nodes
vector<ll> dist(n + 1, 0);
vector<int> parent(n + 1, -1);
int last = -1;

for (int iter = 0; iter < n; iter++) {
  last = -1;
  for (auto& e : edges) {
    if (dist[e.a] + e.c < dist[e.b]) {
      dist[e.b] = dist[e.a] + e.c;
      parent[e.b] = e.a;
      last = e.b;
    }
  }
}

if (last == -1) { cout << "NO"; return 0; }

// Walk back n steps to guarantee we land on the cycle
int v = last;
for (int i = 0; i < n; i++) v = parent[v];

vector<bool> visited(n + 1, false);
vector<int> cycle;
int cur = v;
while (!visited[cur]) {
  visited[cur] = true;
  cycle.push_back(cur);
  cur = parent[cur];
}
cycle.push_back(cur);            // close with v
reverse(cycle.begin(), cycle.end());

cout << "YES\n";
```

```
for (int x : cycle) cout << x << " ";
```

Course Schedule (CSES). Kahn's topological sort: push indegree-0 nodes, decrement neighbors on pop; if order size $< n$ output IMPOSSIBLE.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
vector<int> indeg(n + 1, 0);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  indeg[b]++;
}

queue<int> q;
for (int i = 1; i <= n; i++)
  if (indeg[i] == 0) q.push(i);

vector<int> order;
while (!q.empty()) {
  int u = q.front(); q.pop();
  order.push_back(u);
  for (int v : adj[u])
    if (--indeg[v] == 0) q.push(v);
}

if ((int)order.size() < n) { cout << "IMPOSSIBLE"; return 0; }
for (int x : order) cout << x << " ";
```

Conservation (Kattis). Each process runs in lab 0 or 1; do a 2-queue topo sort starting from each lab and take min forced+voluntary switches.

```
int T;
cin >> T;
while (T--) {
  int n, m;
  cin >> n >> m;

  vector<int> lab(n + 1);
  for (int i = 1; i <= n; i++) cin >> lab[i];

  vector<vector<int>> adj(n + 1);
  vector<int> indeg(n + 1, 0);
  for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    adj[a].push_back(b);
    indeg[b]++;
  }

  auto simulate = [&](int start_lab) {
    vector<int> ind = indeg;
    queue<int> qq[2];
    for (int i = 1; i <= n; i++)
      if (ind[i] == 0) qq[lab[i] - 1].push(i);

    int cur = start_lab, switches = 0;
    while (qq[0].size() + qq[1].size() > 0) {
      if (qq[cur].empty()) { cur ^= 1; switches++; }
      while (!qq[cur].empty()) {
        int u = qq[cur].front(); qq[cur].pop();
        for (int v : adj[u])
          if (--ind[v] == 0) qq[lab[v] - 1].push(v);
      }
      if (!qq[cur ^ 1].empty()) { cur ^= 1; switches++; }
    }
    return switches;
  };

  cout << min(simulate(0), simulate(1)) << "\n";
}
```

Game Routes (CSES). DAG path-count $1 \to n$: Kahn's topo + $ways[v] \mathrel{+}= ways[u] \pmod{10^9+7}$.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1);
vector<int> indeg(n + 1, 0);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  indeg[b]++;
}

const ll MOD = 1e9 + 7;
queue<int> q;
for (int i = 1; i <= n; i++)
  if (indeg[i] == 0) q.push(i);

vector<ll> ways(n + 1, 0);
ways[1] = 1;

while (!q.empty()) {
  int u = q.front(); q.pop();
  for (int v : adj[u]) {
    ways[v] = (ways[v] + ways[u]) % MOD;
    if (--indeg[v] == 0) q.push(v);
  }
}

cout << ways[n];
```

Longest Flight Route (CSES). DAG longest path: topo order + $dist[v] = \max(dist[v], dist[u] + 1)$; reconstruct using the reversed adjacency.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1), radj(n + 1);
vector<int> indeg(n + 1, 0);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  radj[b].push_back(a);
  indeg[b]++;
}

const ll NEG = -1e18;
vector<ll> dist(n + 1, NEG);
dist[1] = 0;

queue<int> q;
for (int i = 1; i <= n; i++)
  if (indeg[i] == 0) q.push(i);

while (!q.empty()) {
```

```
  int u = q.front(); q.pop();
  for (int v : adj[u]) {
    if (dist[u] != NEG)
      dist[v] = max(dist[v], dist[u] + 1);
    if (--indeg[v] == 0) q.push(v);
  }
}

if (dist[n] == NEG) { cout << "IMPOSSIBLE"; return 0; }

// Walk back using distance values
vector<int> path;
int cur = n;
while (cur != 1) {
  path.push_back(cur);
  for (int prev : radj[cur])
    if (dist[prev] == dist[cur] - 1) { cur = prev; break; }
}
path.push_back(1);
reverse(path.begin(), path.end());

cout << path.size() << "\n";
for (int x : path) cout << x << " ";
```

Flight Routes Check (CSES). Kosaraju's SCC; graph is strongly connected iff #SCC=1. Else sample a node from source and sink SCC as counter-example.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1), radj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  radj[b].push_back(a);
}

vector<bool> vis(n + 1, false);
vector<int> order, comp(n + 1, 0);

// Pass 1: iterative DFS on adj, push on exit
auto dfs1 = [&](int s) {
  stack<pair<int,int>> st;
  st.push({s, 0});
  vis[s] = true;
  while (!st.empty()) {
    auto& [u, idx] = st.top();
    if (idx < (int)adj[u].size()) {
      int v = adj[u][idx++];
      if (!vis[v]) { vis[v] = true; st.push({v, 0}); }
    } else {
      order.push_back(u);
      st.pop();
    }
  }
};

// Pass 2: iterative DFS on radj in reverse finish order
auto dfs2 = [&](int s, int c) {
  stack<int> st;
  st.push(s);
  comp[s] = c;
  while (!st.empty()) {
    int u = st.top(); st.pop();
    for (int v : radj[u])
      if (!comp[v]) { comp[v] = c; st.push(v); }
  }
};

for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);

int num_scc = 0;
for (int i = (int)order.size() - 1; i >= 0; i--)
  if (!comp[order[i]]) dfs2(order[i], ++num_scc);

if (num_scc == 1) { cout << "YES"; return 0; }

cout << "NO\n";
int a = -1, b = -1;
for (int i = 1; i <= n; i++) {
  if (comp[i] == num_scc && a == -1) a = i;   // source SCC
  if (comp[i] == 1       && b == -1) b = i;   // sink SCC
}
cout << a << " " << b;
```

Planets and Kingdoms (CSES). Kosaraju's SCC labeling; output number of SCCs and the component id of each node.

```
int n, m;
cin >> n >> m;

vector<vector<int>> adj(n + 1), radj(n + 1);
for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  adj[a].push_back(b);
  radj[b].push_back(a);
}

vector<bool> vis(n + 1, false);
vector<int> order, comp(n + 1, 0);

auto dfs1 = [&](int s) {
  stack<pair<int,int>> st;
  st.push({s, 0});
  vis[s] = true;
  while (!st.empty()) {
    auto& [u, idx] = st.top();
    if (idx < (int)adj[u].size()) {
      int v = adj[u][idx++];
      if (!vis[v]) { vis[v] = true; st.push({v, 0}); }
    } else {
      order.push_back(u);
      st.pop();
    }
  }
};

auto dfs2 = [&](int s, int c) {
  stack<int> st;
  st.push(s);
  comp[s] = c;
  while (!st.empty()) {
    int u = st.top(); st.pop();
    for (int v : radj[u])
      if (!comp[v]) { comp[v] = c; st.push(v); }
  }
};

for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);

int num_scc = 0;
for (int i = (int)order.size() - 1; i >= 0; i--)
  if (!comp[order[i]]) dfs2(order[i], ++num_scc);

cout << num_scc << "\n";
```

```
for (int i = 1; i <= n; i++)
  cout << comp[i] << " \n"[i == n];
```

MST & Disjoint Set Union

Road Reparation (CSES). Kruskal MST: sort edges by cost, unite if different components, accumulate cost; if $< n - 1$ edges used output IMPOSSIBLE.

```
int n, m;
cin >> n >> m;

struct Edge { ll cost; int a, b; };
vector<Edge> edges(m);
for (auto& e : edges) cin >> e.a >> e.b >> e.cost;

sort(edges.begin(), edges.end(),
     [](const Edge& x, const Edge& y) { return x.cost < y.cost;
     });

vector<int> parent(n + 1), sz(n + 1, 1);
iota(parent.begin(), parent.end(), 0);

function<int(int)> find = [&](int x) {
  return parent[x] == x ? x : parent[x] = find(parent[x]);
};

ll total = 0;
int used = 0;
for (auto& e : edges) {
  int ra = find(e.a), rb = find(e.b);
  if (ra == rb) continue;
  if (sz[ra] < sz[rb]) swap(ra, rb);
  parent[rb] = ra;
  sz[ra] += sz[rb];
  total += e.cost;
  if (++used == n - 1) break;
}

if (used < n - 1) cout << "IMPOSSIBLE";
else              cout << total;
```

Lost Map (Kattis). MST on a complete distance matrix with Prim's $O(n^2)$; print parent-child edges as the original tree.

```
int n;
cin >> n;

vector<vector<int>> d(n, vector<int>(n));
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    cin >> d[i][j];

vector<bool> inMST(n, false);
vector<int> minDist(n, INT_MAX);
vector<int> parent(n, -1);
minDist[0] = 0;

for (int iter = 0; iter < n; iter++) {
  int u = -1;
  for (int i = 0; i < n; i++)
    if (!inMST[i] && (u == -1 || minDist[i] < minDist[u]))
      u = i;

  inMST[u] = true;
  if (parent[u] != -1)
    cout << (parent[u] + 1) << " " << (u + 1) << "\n";
  for (int v = 0; v < n; v++)
    if (!inMST[v] && d[u][v] < minDist[v]) {
      minDist[v] = d[u][v];
      parent[v] = u;
    }
}
```

Road Construction (CSES). DSU; after each unite output the current number of components and the size of the largest one.

```
int n, m;
cin >> n >> m;

vector<int> parent(n + 1), sz(n + 1, 1);
iota(parent.begin(), parent.end(), 0);

function<int(int)> find = [&](int x) {
  return parent[x] == x ? x : parent[x] = find(parent[x]);
};

int num_components = n, max_size = 1;

for (int i = 0; i < m; i++) {
  int a, b;
  cin >> a >> b;
  a = find(a); b = find(b);
  if (a != b) {
    if (sz[a] < sz[b]) swap(a, b);
    parent[b] = a;
    sz[a] += sz[b];
    num_components--;
    max_size = max(max_size, sz[a]);
  }
  cout << num_components << " " << max_size << "\n";
}
```

Union-Find (Kattis). Straight DSU: '=' unites, '?' prints yes/no for same-set.

```
int n, m;
cin >> n >> m;

vector<int> parent(n), sz(n, 1);
iota(parent.begin(), parent.end(), 0);

function<int(int)> find = [&](int x) {
  return parent[x] == x ? x : parent[x] = find(parent[x]);
};

while (m--) {
  char op;
  int a, b;
  cin >> op >> a >> b;
  if (op == '=') {
    a = find(a); b = find(b);
    if (a != b) {
      if (sz[a] < sz[b]) swap(a, b);
      parent[b] = a;
      sz[a] += sz[b];
    }
  } else {
    cout << (find(a) == find(b) ? "yes" : "no") << "\n";
  }
}
```

Ladice (Kattis). DSU where each component tracks items vs. capacity; same-root insert if capacity remains, else merge then try; past capacity output SMECE.

```
int n, k;
cin >> n >> k;

vector<int> parent(k + 1), sz(k + 1, 1), items(k + 1, 0);
iota(parent.begin(), parent.end(), 0);

function<int(int)> find = [&](int x) {
  return parent[x] == x ? x : parent[x] = find(parent[x]);
};

for (int i = 0; i < n; i++) {
  int a, b;
  cin >> a >> b;
  int ra = find(a), rb = find(b);

  if (ra == rb) {
    if (items[ra] < sz[ra]) { items[ra]++; cout << "LADICA\n"; }
    else                    cout << "SMECE\n";
  } else {
    if (sz[ra] < sz[rb]) swap(ra, rb);
    parent[rb] = ra;
    sz[ra]    += sz[rb];
    items[ra] += items[rb];
    if (items[ra] < sz[ra]) { items[ra]++; cout << "LADICA\n"; }
    else                    cout << "SMECE\n";
  }
}
```

Almost Union-Find (Kattis). DSU with a "move" op: detach p from its root (decrement size/sum), give p a fresh proxy node, merge into q's set. Also supports size+sum queries.

```
int n, m;
while (cin >> n >> m) {
  int maxNodes = 2 * n + m + 1;
  vector<int> parent(maxNodes + 1), sz(maxNodes + 1, 0);
  vector<ll>  sum(maxNodes + 1, 0);
  vector<int> id(n + 1);

  // id[i] = current proxy node for element i
  for (int i = 1; i <= n; i++) {
    id[i] = n + i;
    parent[n + i] = n + i;
    sz[n + i] = 1;
    sum[n + i] = i;
  }

  function<int(int)> find = [&](int x) {
    return parent[x] == x ? x : parent[x] = find(parent[x]);
  };
  auto unite = [&](int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);
    parent[b] = a;
    sz[a]  += sz[b];
    sum[a] += sum[b];
  };

  int nextProxy = 2 * n + 1;
  while (m--) {
    int op;
    cin >> op;
    if (op == 1) {
      int p, q;
      cin >> p >> q;
      unite(id[p], id[q]);
    } else if (op == 2) {
      int p, q;
      cin >> p >> q;
      if (find(id[p]) == find(id[q])) continue;
      int rp = find(id[p]);
      sz[rp]--;
      sum[rp] -= p;
      int np = nextProxy++;
      parent[np] = np;
      sz[np]  = 1;
      sum[np] = p;
      id[p]   = np;
      unite(id[p], id[q]);
    } else {
      int p;
      cin >> p;
      int r = find(id[p]);
      cout << sz[r] << " " << sum[r] << "\n";
    }
  }
}
```

Dynamic Programming

Book Shop (CSES). 0/1 knapsack: price = weight, pages = value, capacity x. Items outer, capacity BACKWARD.

```
int n, x;
cin >> n >> x;

vector<int> h(n), s(n);
for (int i = 0; i < n; i++) cin >> h[i];
for (int i = 0; i < n; i++) cin >> s[i];

vector<ll> dp(x + 1, 0);
for (int i = 0; i < n; i++)
  for (int j = x; j >= h[i]; j--)
    dp[j] = max(dp[j], dp[j - h[i]] + s[i]);

cout << dp[x];
```

Coin Combinations I (CSES). Count *ordered* ways to form sum ($1+2 \neq 2+1$). Target outer, coins inner.

```
int n;
ll sum;
cin >> n >> sum;

vector<ll> coins(n);
for (auto& c : coins) cin >> c;

const ll MOD = 1e9 + 7;
vector<ll> dp(sum + 1, 0);
dp[0] = 1;

for (ll s = 1; s <= sum; s++)
  for (ll c : coins)
    if (s - c >= 0)
      dp[s] = (dp[s] + dp[s - c]) % MOD;

cout << dp[sum];
```

Coin Combinations II (CSES). Count *unordered* ways ($1+2 == 2+1$). Coins outer, target inner.

```
int n;
ll sum;
cin >> n >> sum;

vector<ll> coins(n);
```

```
for (auto& c : coins) cin >> c;

const ll MOD = 1e9 + 7;
vector<ll> dp(sum + 1, 0);
dp[0] = 1;

for (ll c : coins)
  for (ll s = c; s <= sum; s++)
    dp[s] = (dp[s] + dp[s - c]) % MOD;

cout << dp[sum];
```

Minimizing Coins (CSES). Min #coins to reach sum (unbounded knapsack); output -1 if unreachable.

```
int n;
ll sum;
cin >> n >> sum;

vector<ll> coins(n);
for (auto& c : coins) cin >> c;

const ll INF = 1e9;
vector<ll> dp(sum + 1, INF);
dp[0] = 0;

for (ll s = 1; s <= sum; s++)
  for (ll c : coins)
    if (s - c >= 0)
      dp[s] = min(dp[s], dp[s - c] + 1);

cout << (dp[sum] >= INF ? -1 : dp[sum]);
```

Dice Combinations (CSES). Count ordered ways to roll dice summing to n; faces 1..6.

```
ll n;
cin >> n;

const ll MOD = 1e9 + 7;
vector<ll> dp(n + 1, 0);
dp[0] = 1;

for (ll i = 1; i <= n; i++)
  for (int f = 1; f <= 6; f++)
    if (i - f >= 0)
      dp[i] = (dp[i] + dp[i - f]) % MOD;

cout << dp[n];
```

Money Sums (CSES). Subset-sum: output all distinct sums achievable. Boolean DP, items outer, sums BACKWARD.

```
int n;
cin >> n;

vector<int> x(n);
int maxSum = 0;
for (int& v : x) { cin >> v; maxSum += v; }

vector<bool> dp(maxSum + 1, false);
dp[0] = true;

for (int i = 0; i < n; i++)
  for (int s = maxSum; s >= x[i]; s--)
    if (dp[s - x[i]]) dp[s] = true;

vector<int> result;
for (int s = 1; s <= maxSum; s++)
  if (dp[s]) result.push_back(s);

cout << result.size() << "\n";
for (int i = 0; i < (int)result.size(); i++) {
  cout << result[i];
  if (i + 1 < (int)result.size()) cout << " ";
}
```

Grid Paths I (CSES). Count paths in grid moving right/down, blocked '*' cells: $dp[i][j] = dp[i-1][j] + dp[i][j-1] \bmod 10^9 + 7$.

```
int n;
cin >> n;

vector<vector<char>> grid(n, vector<char>(n, '.'));
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    cin >> grid[i][j];

const int MOD = 1e9 + 7;
vector<vector<int>> dp(n, vector<int>(n, 0));
dp[0][0] = (grid[0][0] == '*' ? 0 : 1);

for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++) {
    if (grid[i][j] == '*') { dp[i][j] = 0; continue; }
    if (i > 0) dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD;
    if (j > 0) dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD;
  }

cout << dp[n - 1][n - 1];
```

Rectangle Cutting (CSES). Min cuts to divide $A \times B$ into unit-side squares. Try every horizontal and vertical split.

```
int A, B;
cin >> A >> B;

vector<vector<int>> dp(A + 1, vector<int>(B + 1, 0));
for (int i = 1; i <= A; i++)
  for (int j = 1; j <= B; j++) {
    if (i == j) { dp[i][j] = 0; continue; }
    dp[i][j] = INT_MAX;
    // Horizontal cuts (only up to i/2 by symmetry)
    for (int k = 1; k <= i / 2; k++)
      dp[i][j] = min(dp[i][j], 1 + dp[k][j] + dp[i - k][j]);
    // Vertical cuts
    for (int k = 1; k <= j / 2; k++)
      dp[i][j] = min(dp[i][j], 1 + dp[i][k] + dp[i][j - k]);
  }

cout << dp[A][B];
```

Removing Digits (CSES). Min steps to reach 0 by subtracting a non-zero digit of the current number; $dp[i] = 1 + \min_{d \in digits(i),\, d>0} dp[i-d]$.

```
ll n;
cin >> n;

vector<ll> dp(n + 1, 1e9);
dp[0] = 0;

for (ll i = 1; i <= n; i++) {
  string s = to_string(i);
  for (char c : s) {
    int digit = c - '0';
```

```
    if (digit != 0)
      dp[i] = min(dp[i], dp[i - digit] + 1);
  }
}

cout << dp[n];
```

Edit Distance (CSES). Levenshtein distance. Base $dp[i][0] = i, dp[0][j] = j$; match $\rightarrow$ copy diag, else $1 + \min(\text{del, ins, rep})$.

```
string a, b;
cin >> a >> b;
int n = a.size(), m = b.size();

vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 0; i <= n; i++) dp[i][0] = i;   // delete all of a
for (int j = 0; j <= m; j++) dp[0][j] = j;   // insert all of b

for (int i = 1; i <= n; i++)
  for (int j = 1; j <= m; j++) {
    if (a[i - 1] == b[j - 1])
      dp[i][j] = dp[i - 1][j - 1];
    else
      dp[i][j] = 1 + min({dp[i - 1][j],         // delete
                          dp[i][j - 1],         // insert
                          dp[i - 1][j - 1]});   // replace
  }

cout << dp[n][m];
```

Array Description (CSES). Count arrays of length n (values $1..m$, neighbors differ by ≤ 1) matching fixed non-zero positions; rolling 1D DP.

```
int n, m;
cin >> n >> m;

vector<int> x(n);
for (int i = 0; i < n; i++) cin >> x[i];

const int MOD = 1e9 + 7;
vector<ll> dp(m + 1, 0);

// Initialize first position
if (x[0] == 0) for (int v = 1; v <= m; v++) dp[v] = 1;
else           dp[x[0]] = 1;

for (int i = 1; i < n; i++) {
  vector<ll> ndp(m + 1, 0);
  for (int v = 1; v <= m; v++) {
    if (x[i] != 0 && x[i] != v) continue;
    for (int prev = v - 1; prev <= v + 1; prev++) {
      if (prev < 1 || prev > m) continue;
      ndp[v] = (ndp[v] + dp[prev]) % MOD;
    }
  }
  dp = ndp;
}

ll ans = 0;
for (int v = 1; v <= m; v++) ans = (ans + dp[v]) % MOD;
cout << ans;
```

Counting Towers (CSES). Pre-compute $f[n]$ via $f[n] = 6f[n-1] - 7f[n-2]$ (with $f[1]$=2, $f[2]$=8); answer each query in O(1).

```
const int MOD = 1e9 + 7;
const int MAXN = 1e6 + 1;

vector<ll> f(MAXN);
f[1] = 2;
f[2] = 8;
for (int i = 3; i < MAXN; i++)
  f[i] = (6 * f[i - 1] % MOD - 7 * f[i - 2] % MOD + MOD) % MOD;

int t;
cin >> t;
while (t--) {
  int n;
  cin >> n;
  cout << f[n] << "\n";
}
```

Tri Tiling (Kattis). Tilings of $3 \times n$ with dominoes. Odd $n \rightarrow 0$; else $f(n) = 4f(n-2) - f(n-4)$, $f(0)$=1, $f(2)$=3.

```
// f(0)=1, f(2)=3, f(n) = 4*f(n-2) - f(n-4);  f(odd) = 0
ll dp[31] = {};
dp[0] = 1;
dp[2] = 3;
for (int i = 4; i <= 30; i += 2)
  dp[i] = 4 * dp[i - 2] - dp[i - 4];

int n;
while (cin >> n && n != -1) {
  if (n % 2 == 1) cout << 0 << "\n";
  else            cout << dp[n] << "\n";
}
```

Nine Packs (Kattis). Two independent 0/1 subset-sum DPs; find smallest positive sum reachable by both and minimize total #packs.

```
int nh;
cin >> nh;
vector<int> hdogs(nh);
int maxH = 0;
for (int i = 0; i < nh; i++) { cin >> hdogs[i]; maxH += hdogs[i
    ]; }

int nb;
cin >> nb;
vector<int> buns(nb);
int maxB = 0;
for (int i = 0; i < nb; i++) { cin >> buns[i]; maxB += buns[i];
    }

const int INF = INT_MAX / 2;
vector<int> hdp(maxH + 1, INF);
vector<int> bdp(maxB + 1, INF);
hdp[0] = 0;
bdp[0] = 0;

// 0/1 knapsack, min count, each pack used at most once
for (int i = 0; i < nh; i++)
  for (int s = maxH; s >= hdogs[i]; s--)
    if (hdp[s - hdogs[i]] != INF)
      hdp[s] = min(hdp[s], hdp[s - hdogs[i]] + 1);

for (int i = 0; i < nb; i++)
  for (int s = maxB; s >= buns[i]; s--)
    if (bdp[s - buns[i]] != INF)
      bdp[s] = min(bdp[s], bdp[s - buns[i]] + 1);
```

```
int ans = INF;
int limit = min(maxH, maxB);
for (int s = 1; s <= limit; s++)
  if (hdp[s] != INF && bdp[s] != INF)
    ans = min(ans, hdp[s] + bdp[s]);

if (ans == INF) cout << "impossible";
else            cout << ans;
```

Exact Change (Kattis). 0/1 knapsack min-count; find smallest sum $\geq$ price that is reachable, with fewest coins.

```
int T;
cin >> T;
while (T--) {
  int price, n;
  cin >> price >> n;

  vector<int> coins(n);
  int total = 0;
  for (int i = 0; i < n; i++) { cin >> coins[i]; total += coins[
     i]; }

  vector<int> dp(total + 1, INT_MAX);
  dp[0] = 0;

  // 0/1 knapsack, right-to-left (each coin at most once)
  for (int i = 0; i < n; i++)
    for (int s = total; s >= coins[i]; s--)
      if (dp[s - coins[i]] != INT_MAX)
        dp[s] = min(dp[s], dp[s - coins[i]] + 1);

  // Smallest reachable sum >= price
  for (int s = price; s <= total; s++)
    if (dp[s] != INT_MAX) {
      cout << s << " " << dp[s] << "\n";
      break;
    }
}
```

Cent Savings (Kattis). Prefix-sum DP with k dividers: $dp_g[i] = \min_m dp_{g-1}[m] + \text{round}_{10}(pre[i] - pre[m])$ over $g = 1..k{+}1$.

```
int n, k;
cin >> n >> k;

vector<int> pre(n + 1, 0);
for (int i = 1; i <= n; i++) {
  int x;
  cin >> x;
  pre[i] = pre[i - 1] + x;
}

auto round10 = [](int x) { return ((x + 5) / 10) * 10; };

int maxGroups = min(k + 1, n);
const int INF = INT_MAX / 2;

vector<int> prev_dp(n + 1, INF);
prev_dp[0] = 0;

int ans = INF;
for (int g = 1; g <= maxGroups; g++) {
  vector<int> cur_dp(n + 1, INF);
  for (int i = 1; i <= n; i++)
    for (int m = g - 1; m < i; m++) {
      if (prev_dp[m] == INF) continue;
      int cost = round10(pre[i] - pre[m]);
      cur_dp[i] = min(cur_dp[i], prev_dp[m] + cost);
    }
  ans = min(ans, cur_dp[n]);
  prev_dp = cur_dp;
}

cout << ans;
```

Restaurant Orders (Kattis). Unbounded knapsack counting distinct multisets (cap at 2): $0 \to$ Impossible, $\geq 2 \to$ Ambiguous, else reconstruct greedily.

```
int n;
cin >> n;
vector<int> price(n + 1);
for (int i = 1; i <= n; i++) cin >> price[i];

int q;
cin >> q;
vector<int> orders(q);
int maxOrder = 0;
for (int i = 0; i < q; i++) {
  cin >> orders[i];
  maxOrder = max(maxOrder, orders[i]);
}

// Unbounded knapsack counting multisets, capped at 2
vector<int> ways(maxOrder + 1, 0);
ways[0] = 1;
for (int i = 1; i <= n; i++)
  for (int s = price[i]; s <= maxOrder; s++)
    ways[s] = min(2, ways[s] + ways[s - price[i]]);

for (int i = 0; i < q; i++) {
  int target = orders[i];
  if (ways[target] == 0) {
    cout << "Impossible\n";
  } else if (ways[target] >= 2) {
    cout << "Ambiguous\n";
  } else {
    // Reconstruct the unique multiset greedily
    vector<int> items;
    int s = target;
    while (s > 0) {
      for (int j = 1; j <= n; j++) {
        if (price[j] <= s && ways[s - price[j]] == 1) {
          items.push_back(j);
          s -= price[j];
          break;
        }
      }
    }
    for (int j = 0; j < (int)items.size(); j++)
      cout << items[j] << " \n"[j + 1 == (int)items.size()];
  }
}
```

Segment Tree Problems

Static Sum Range Queries (CSES). Build iterative seg tree for sum; answer range-sum queries only (no updates).

```
class SegTree {
public:
  vector<ll> tree;
  int n;
  SegTree(vector<ll>& a) {
    n = a.size();
    tree.assign(2 * n, 0);
```

```
    for (int i = 0; i < n; i++) tree[i + n] = a[i];
    for (int i = n - 1; i > 0; i--)
      tree[i] = tree[2 * i] + tree[2 * i + 1];
  }
  ll query(int l, int r) {           // [l, r)
    ll ans = 0;
    l += n; r += n;
    while (l < r) {
      if (l & 1) ans += tree[l++];
      if (r & 1) ans += tree[--r];
      l >>= 1; r >>= 1;
    }
    return ans;
  }
};

int n, q;
cin >> n >> q;
vector<ll> a(n);
for (auto& v : a) cin >> v;

SegTree st(a);
while (q--) {
  int l, r;
  cin >> l >> r;
  cout << st.query(l - 1, r) << "\n";
}
```

Static Minimum Range Queries (CSES). Same iterative skeleton, but combine = min; identity = $+\infty$.

```
class SegTree {
public:
  vector<ll> tree;
  int n;
  SegTree(vector<ll>& a) {
    n = a.size();
    tree.assign(2 * n, (ll)1e18);
    for (int i = 0; i < n; i++) tree[i + n] = a[i];
    for (int i = n - 1; i > 0; i--)
      tree[i] = min(tree[2 * i], tree[2 * i + 1]);
  }
  ll query(int l, int r) {
    ll ans = 1e18;
    l += n; r += n;
    while (l < r) {
      if (l & 1) ans = min(ans, tree[l++]);
      if (r & 1) ans = min(ans, tree[--r]);
      l >>= 1; r >>= 1;
    }
    return ans;
  }
};

int n, q;
cin >> n >> q;
vector<ll> a(n);
for (auto& v : a) cin >> v;

SegTree st(a);
while (q--) {
  int l, r;
  cin >> l >> r;
  cout << st.query(l - 1, r) << "\n";
}
```

Dynamic Sum Range Queries (CSES). Sum seg tree with point updates: set $a[i] = v$, climb from leaf rebuilding parents.

```
class SegTree {
public:
  vector<ll> tree;
  int n;
  SegTree(vector<ll>& a) {
    n = a.size();
    tree.assign(2 * n, 0);
    for (int i = 0; i < n; i++) tree[i + n] = a[i];
    for (int i = n - 1; i > 0; i--)
      tree[i] = tree[2 * i] + tree[2 * i + 1];
  }
  void update(int i, ll v) {
    i += n;
    tree[i] = v;
    while (i > 1) {
      i /= 2;
      tree[i] = tree[2 * i] + tree[2 * i + 1];
    }
  }
  ll query(int l, int r) {
    ll ans = 0;
    l += n; r += n;
    while (l < r) {
      if (l & 1) ans += tree[l++];
      if (r & 1) ans += tree[--r];
      l >>= 1; r >>= 1;
    }
    return ans;
  }
};

int n, q;
cin >> n >> q;
vector<ll> a(n);
for (auto& v : a) cin >> v;

SegTree st(a);
while (q--) {
  int op;
  cin >> op;
  if (op == 1) {
    int i;
    ll v;
    cin >> i >> v;
    st.update(i - 1, v);
  } else {
    int l, r;
    cin >> l >> r;
    cout << st.query(l - 1, r) << "\n";
  }
}
```

Dynamic Minimum Range Queries (CSES). Min seg tree with point updates; same skeleton, replace + with `min` and identity with 1e18.

```
class SegTree {
public:
  vector<ll> tree;
  int n;
  SegTree(vector<ll>& a) {
    n = a.size();
    tree.assign(2 * n, (ll)1e18);
    for (int i = 0; i < n; i++) tree[i + n] = a[i];
    for (int i = n - 1; i > 0; i--)
      tree[i] = min(tree[2 * i], tree[2 * i + 1]);
  }
  void update(int i, ll v) {
    i += n;
```

```
    tree[i] = v;
    while (i > 1) {
      i /= 2;
      tree[i] = min(tree[2 * i], tree[2 * i + 1]);
    }
  }
  ll query(int l, int r) {
    ll ans = 1e18;
    l += n; r += n;
    while (l < r) {
      if (l & 1) ans = min(ans, tree[l++]);
      if (r & 1) ans = min(ans, tree[--r]);
      l >>= 1; r >>= 1;
    }
    return ans;
  }
};

int n, q;
cin >> n >> q;
vector<ll> a(n);
for (auto& v : a) cin >> v;

SegTree st(a);
while (q--) {
  int op;
  cin >> op;
  if (op == 1) {
    int i;
    ll v;
    cin >> i >> v;
    st.update(i - 1, v);
  } else {
    int l, r;
    cin >> l >> r;
    cout << st.query(l - 1, r) << "\n";
  }
}
```

Range Update Queries (CSES). Difference-array trick: on diff $d[i] = a[i] - a[i-1]$, range-add u on $[l, r]$ = point-add $+u$ at l and $-u$ at $r+1$. Point query $a[k] = \sum_{i=0}^{k} d[i]$.

```
// Power-of-2 sum seg tree over the difference array.
class SegTree {
public:
  vector<ll> tree;
  ll n, sz;
  SegTree(vector<ll>& a) {
    sz = a.size();
    n = 1;
    while (n < sz) n <<= 1;
    tree.assign(2 * n, 0);
    for (ll i = 0; i < sz; i++) tree[i + n] = a[i];
    for (ll i = n - 1; i > 0; i--)
      tree[i] = tree[2 * i] + tree[2 * i + 1];
  }
  void add(ll i, ll u) {
    i += n;
    tree[i] += u;
    while (i > 1) {
      i /= 2;
      tree[i] = tree[2 * i] + tree[2 * i + 1];
    }
  }
  ll query(ll k) {                    // a[k] = sum of diff over
     [0, k]
    ll ans = 0, l = n, r = k + n + 1;
    while (l < r) {
      if (l & 1) ans += tree[l++];
      if (r & 1) ans += tree[--r];
      l >>= 1; r >>= 1;
    }
    return ans;
  }
};

int n, q;
cin >> n >> q;
vector<ll> diff(n);
ll prev = 0;
for (ll i = 0; i < n; i++) {
  ll x;
  cin >> x;
  diff[i] = x - prev;
  prev = x;
}

SegTree st(diff);
while (q--) {
  int op;
  cin >> op;
  if (op == 1) {
    ll l, r, u;
    cin >> l >> r >> u;
    st.add(l - 1, u);
    if (r < n) st.add(r, -u);
  } else {
    ll k;
    cin >> k;
    cout << st.query(k - 1) << "\n";
  }
}
```

Math / Number Theory / Misc

Counting Divisors (CSES). For each x, loop $j = 2..\sqrt{x}$; every divisor j contributes 2, unless $j^2 = x$ (contributes 1). Start count at 2 for 1 and x; special-case $x = 1$.

```
int n;
cin >> n;

for (int t = 0; t < n; t++) {
  int x;
  cin >> x;
  if (x == 1) { cout << 1 << "\n"; continue; }

  int count = 2;                        // 1 and x
  for (int j = 2; j * j <= x; j++) {
    if (j * j == x)       count++;
    else if (x % j == 0)  count += 2;
  }
  cout << count << "\n";
}
```

Prime Sieve (Kattis). Classic Eratosthenes sieve up to n; answer count of primes plus 0/1 per query.

```
int n, q;
cin >> n >> q;

vector<bool> is_prime(n + 1, true);
is_prime[0] = is_prime[1] = false;

for (int i = 2; i * i <= n; i++)
  if (is_prime[i])
```

```
    for (int j = i * i; j <= n; j += i)
      is_prime[j] = false;

int count = 0;
for (int i = 2; i <= n; i++) if (is_prime[i]) count++;
cout << count << "\n";

while (q--) {
  int x;
  cin >> x;
  cout << (is_prime[x] ? 1 : 0) << "\n";
}
```

Modular Arithmetic (Kattis). $(a \pm b) \bmod p$ and $ab \bmod p$ trivial; division $a/b \bmod p = a \cdot b^{-1}$ via extended Euclid; output -1 if $\gcd(b, p) \neq 1$.

```
function<ll(ll,ll,ll&,ll&)> extgcd =
  [&](ll a, ll b, ll& x, ll& y) -> ll {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
  };

auto modinv = [&](ll a, ll m) -> ll {
  ll x, y;
  ll g = extgcd(a, m, x, y);
  if (g != 1) return -1;              // no inverse
  return (x % m + m) % m;
};

ll p, n;
while (cin >> p >> n && (p || n)) {
  while (n--) {
    ll a, b;
    char op;
    cin >> a >> op >> b;
    a = ((a % p) + p) % p;
    b = ((b % p) + p) % p;

    if      (op == '+') cout << (a + b) % p        << "\n";
    else if (op == '-') cout << (a - b + p) % p    << "\n";
    else if (op == '*') cout << a * b % p           << "\n";
    else {
      if (b == 0) { cout << "-1\n"; continue; }
      ll inv = modinv(b, p);
      if (inv == -1) cout << "-1\n";
      else           cout << a * inv % p << "\n";
    }
  }
}
```

GCDs (Kattis). Count distinct subarray GCDs. Key fact: the set of GCDs of subarrays ending at i has O(log) values; propagate via gcd-ing with $a[i]$.

```
int n;
cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];

set<int> all_gcds;
vector<int> prev;                       // distinct GCDs of
    subarrays ending at i-1
for (int i = 0; i < n; i++) {
  vector<int> curr = {a[i]};            // subarray [i..i]
  for (int g : prev) {
    int ng = __gcd(g, a[i]);
    if (curr.back() != ng) curr.push_back(ng);
  }
  for (int g : curr) all_gcds.insert(g);
  prev = curr;
}

cout << all_gcds.size();
```

Grandpa Bernie (Kattis). `map<string,vector<int>>` of each country's trip years; sort each list once; answer k-th smallest in O(1).

```
int n;
cin >> n;
map<string, vector<int>> trips;
for (int i = 0; i < n; i++) {
  string country;
  int year;
  cin >> country >> year;
  trips[country].push_back(year);
}

for (auto& p : trips)
  sort(p.second.begin(), p.second.end());

int q;
cin >> q;
while (q--) {
  string country;
  int k;
  cin >> country >> k;
  cout << trips[country][k - 1] << "\n";
}
```

Baby Bites (Kattis). Read n tokens; each is either "mumble" or a base-10 integer; verify it equals its position. Output "something is fishy" on first mismatch, else "makes sense".

```
int n;
cin >> n;
for (int i = 1; i <= n; i++) {
  string a;
  cin >> a;
  if (a != "mumble" && stoi(a) != i) {
    cout << "something is fishy";
    return 0;
  }
}
cout << "makes sense";
```